Experiment.8

Aim:

To implement and demonstrate the working of Service Worker events like fetch, sync, and push in an E-commerce Progressive Web Application (PWA).

Theory:

Service Worker:

A Service Worker is a script that runs in the background of the browser, separate from the main thread. It acts as a programmable network proxy and is a core technology used in Progressive Web Applications (PWAs). Service workers enable features such as offline support, background data synchronization, and push notifications, thereby enhancing the reliability and functionality of web applications.

Key Characteristics of Service Workers:

- Background Operation: They continue to work even when the application is not active.
- HTTPS Requirement: They only run on secure origins to ensure user safety.
- Event-Driven Architecture: All operations are event-based and rely heavily on Promises for asynchronous handling.
- No DOM Access: Service workers operate independently and interact with the web page using the postMessage API.

Fetch Event:

The fetch event is triggered every time the web application makes a network request. The service worker can intercept these requests and respond with cached content or fetch fresh data from the network.

Common Strategies:

1. Cache First:
    - Checks the cache before making a network request.
    - Returns cached response if available; otherwise fetches from the network.
2. Network First:
    - Tries to fetch the resource from the network.
    - Falls back to the cached version if the network is unavailable.

Sync Event (Background Synchronization):

The sync event allows the service worker to defer actions until the user is online. It is useful for tasks such as submitting data that was generated while the user was offline.

Typical Workflow:

1. The user initiates an action (e.g., form submission) while offline.
2. Data is saved locally, usually in IndexedDB.
3. A sync event is registered using registration.sync.register('sync-data').
4. Once the connection is restored, the service worker is activated and performs the pending task.

Push Event:

The push event is fired when the browser receives a push message from a server. This allows the service worker to display a notification, even when the application is not currently in use.

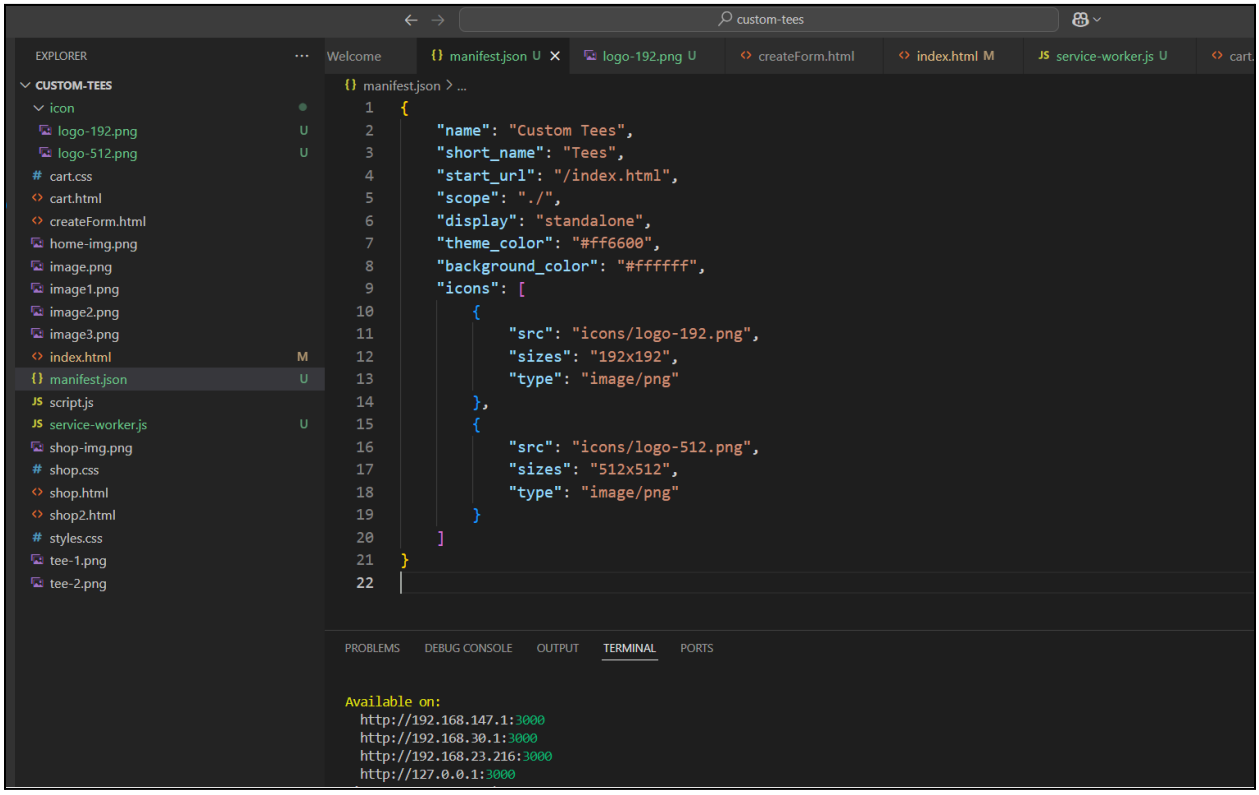
Workflow:

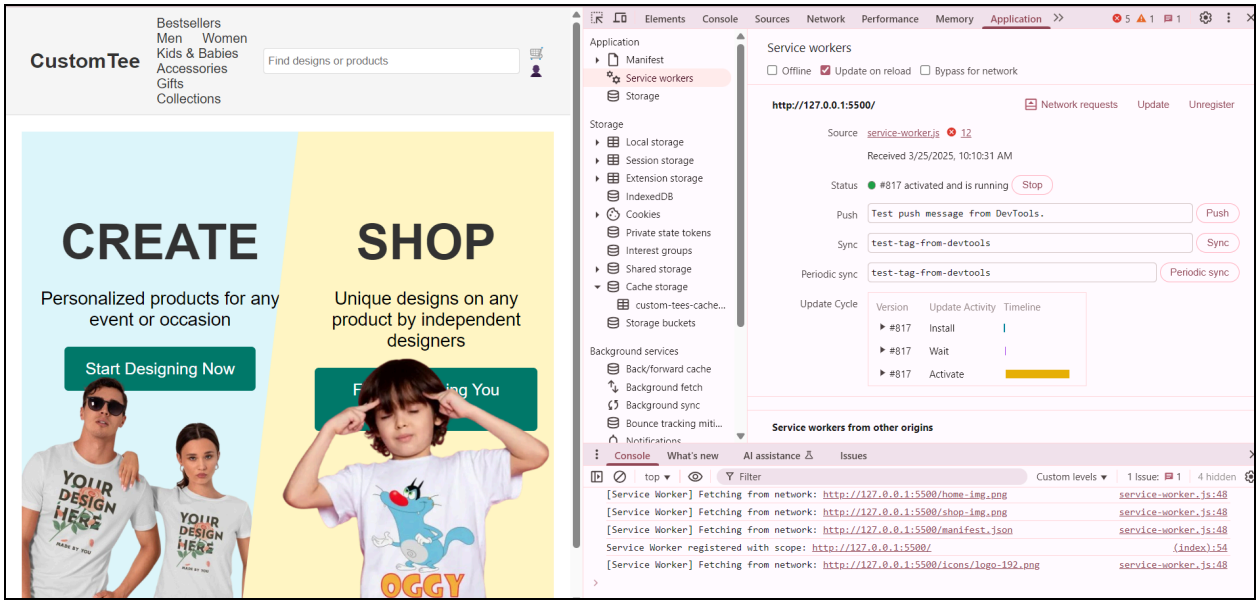
1. The client subscribes to a push service using the Push API.

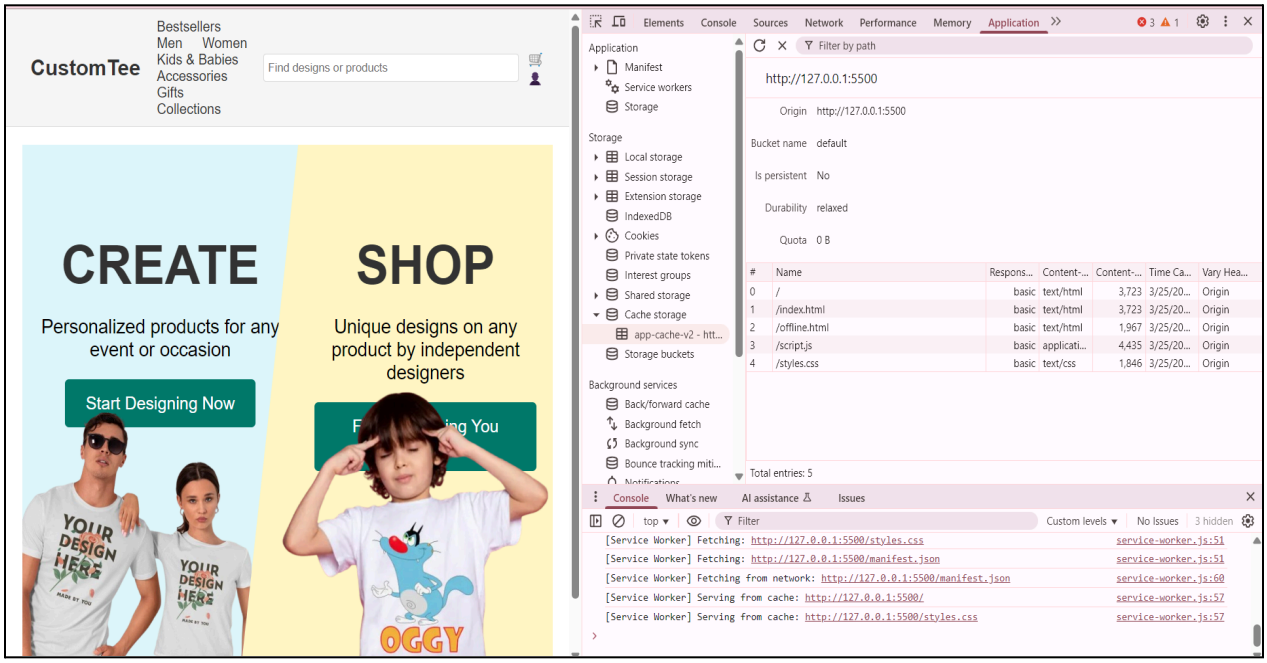
2. The server sends a push message to the subscribed client.

3. The service worker handles the push event and displays a notification using the Notification API.

Output:







Conclusion:

Service workers are an essential part of modern web applications, especially PWAs. They provide the ability to work offline, delay background tasks until a network is available, and engage users with real-time notifications. The fetch, sync, and push events enable powerful features that enhance user experience, reliability, and interactivity of web apps, even in challenging network conditions.